Data Structures Using C

UNIT 1

GEETHA N

ASSISTANT PROFESSOR

B.M.S. COLLEGE OF ENGINEERING

Introduction : Data and Information

What is Data?

- Raw facts used for reference or analysis.
- Set of values of qualitative and quantitative data.
- Numbers, characters, symbols, images ets., which can be processed by a computer.
- Data must be interpreted, by a human or machine, to derive meaning.
- So data is meaningless.
- Conversion of raw data into machine readable form is data processing.

Data Examples

- Yes, Yes, No, Yes, No, Yes, No, Yes
- 42, 63, 96, 74, 56, 86
- 111192, 111245

- None of the above data sets have any meaning until they are given a CONTEXT and PROCESSED into a usable form.

Example 1

- For example: what does the number 29061996 mean ?
- Is it :
  - ★ A birthdate? (29th June 1996)
  - ★ A bank account number?
  - ★ A club membership number?
  - ★ A telephone number?
- Without processing or more information this data is meaningless.

Example 2

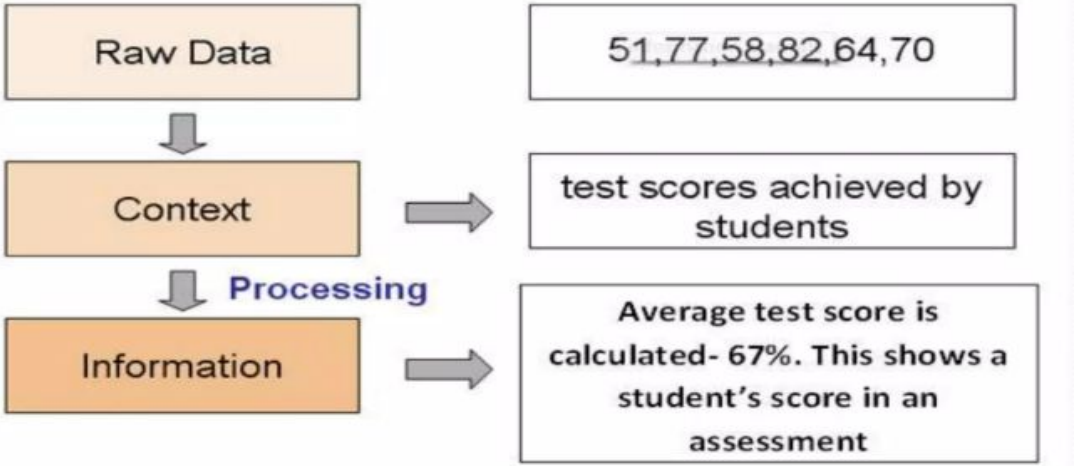
- The colour red is data as it has no meaning.
- What information we attach to the red color is information,
- For example:
  - ★ On a traffic light it means STOP.
  - ★ In a football match a red card means that the player has been suspended.
  - ★ In certain places the red color may mean a danger zone.

Data can take many forms

- Alphanumeric data (combination of numbers and letters)
- Text Data (words, sentences and paragraphs used in communication)
- Image data (Graphics, shapes, figures)
- Audio (Human voice and other sounds)
- Signal (ECG signal)

What is Information

- Data that has been processed within a context to give it meaning.
- “ Information is data that has been processed. “
- “ Information is interpreted data. “
- “ Information is meaningful. (summarized, organized, analysed) “



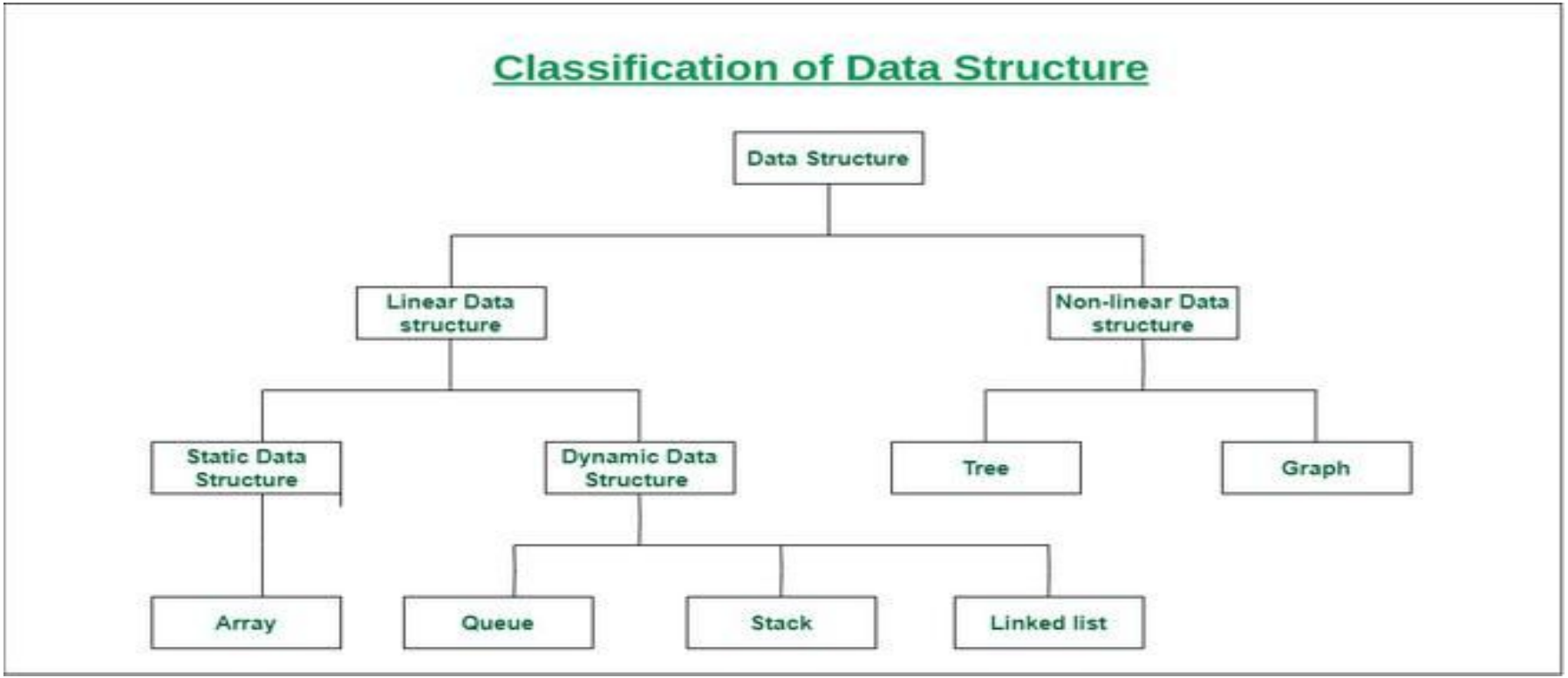
Data Structures

- The way of organizing data in a computer so that it can be used effectively.
- The idea is to reduce the space and time complexity of different tasks.
- It is also used for processing, retrieving, and storing data.

Here is a list of the need for data structures.

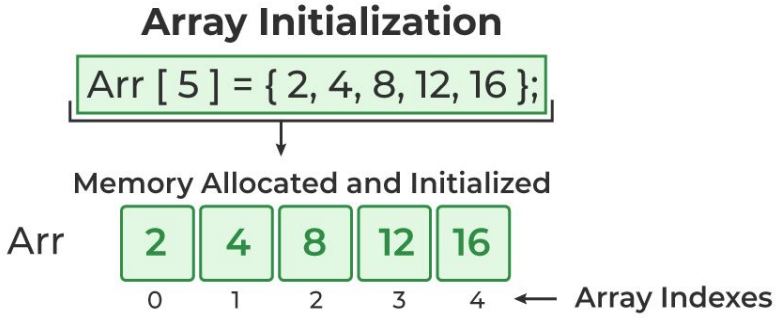
- Data structure modification is easy.
- It requires less time.
- Save storage space.
- Data representation is easy.
- Easy access to the large database.

Classification/ Types of Data Structures



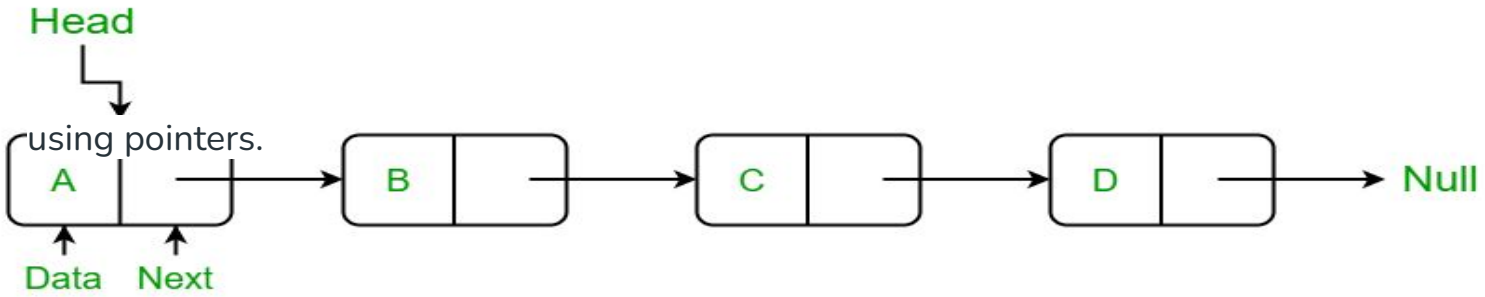
1. Array

- An array is a collection of data items stored at contiguous memory locations.
- The idea is to store multiple items of the same type together.
- Array elements are accessed using array indexes.



Linked List

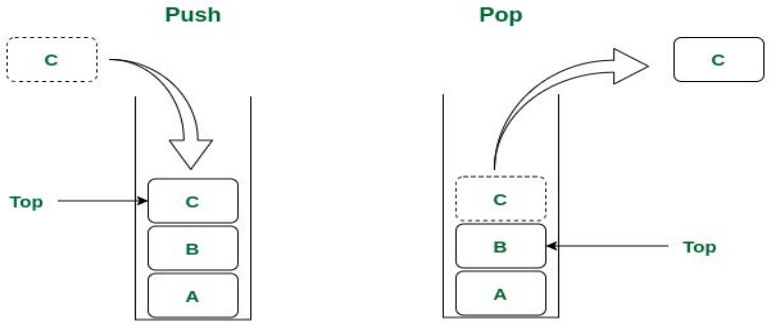
- Like arrays, Linked List is a linear data structure.
- Unlike arrays, linked list elements are not stored at a contiguous location; the elements are linked using pointers.



Linked Data Structure

Stack

- Stack is a linear data structure which follows a particular order in which the operations are performed.
- The order may be LIFO(Last In First Out) or FILO(First In Last Out).
- In stack, all insertion and deletion are permitted at only one end of the list.



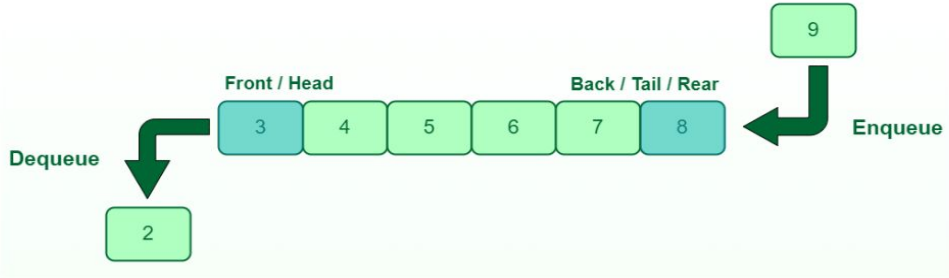
Stack Data Structure

Stack operations

- **push():** When this operation is performed, an element is inserted into the stack.
- **pop():** When this operation is performed, an element is removed from the top of the stack and is returned.
- **top():** This operation will return the last inserted element that is at the top without removing it.
- **size():** This operation will return the size of the stack i.e. the total number of elements present in the stack.
- **isEmpty():** This operation indicates whether the stack is empty or not.

Queue

- Queue is a linear structure which follows a particular order in which the operations are performed.
- The order is First In First Out (FIFO).
- In the queue, items are inserted at one end and deleted from the other end.
- The difference between stacks and queues is in removing. In a stack we remove the item the most recently added; in a queue, we remove the item the least recently added.



Queue Data Structure

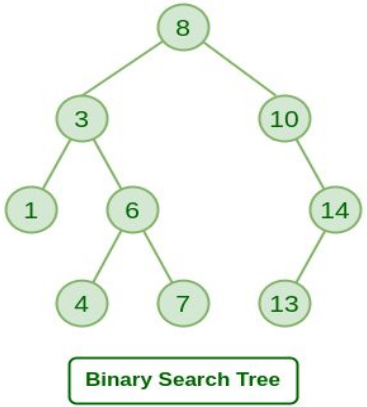
Queue Operations

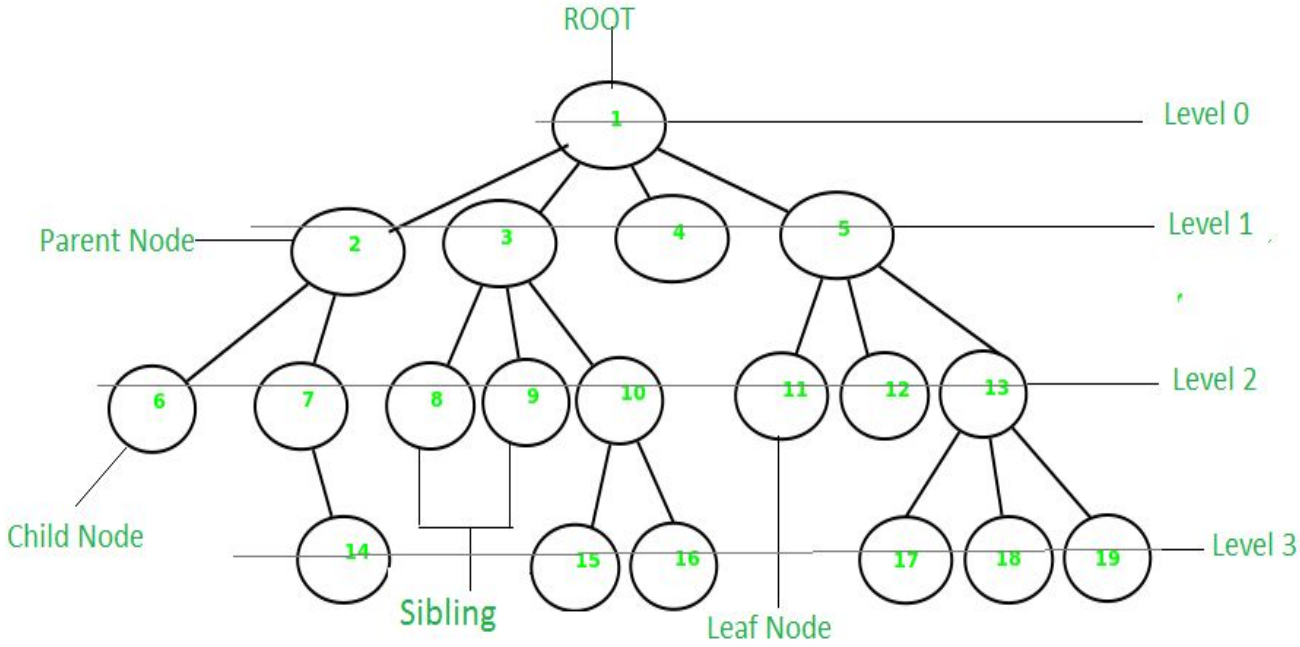
- **Enqueue():** Adds (or stores) an element to the end of the queue..
- **Dequeue():** Removal of elements from the queue.
- **Peek() or front():** Acquires the data element available at the front node of the queue without deleting it.
- **rear():** This operation returns the element at the rear end without removing it.
- **isFull():** Validates if the queue is full.
- **isNull():** Checks if the queue is empty.

Trees

A Binary Search Tree following the additional properties:

- The left part of the root node contains keys less than the root node key.
- The right part of the root node contains keys greater than the root node key.
- There is no duplicate key present in the binary tree.

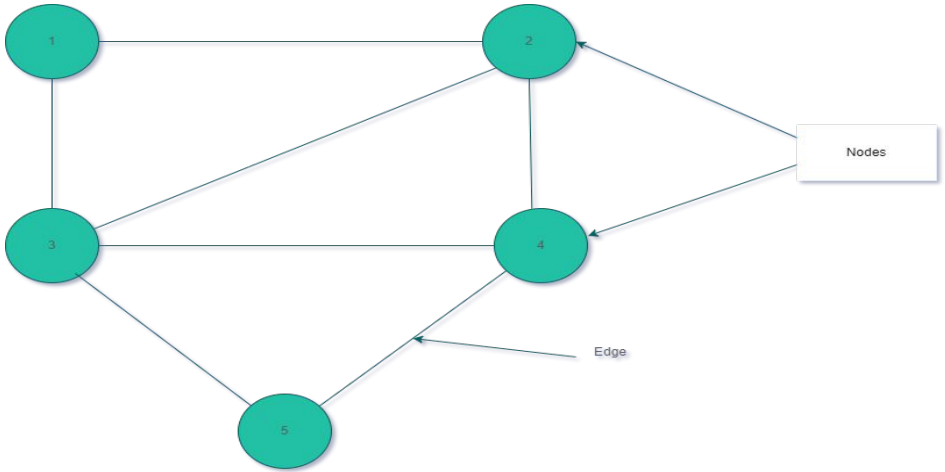



ROOT
1
Level 0
Parent Node
2
3
4
5
Level 1
6
7
8
9
10
11
12
13
Level 2
Child Node
14
15
16
17
18
19
Level 3
Sibling
Leaf Node

Graph

Graph is a data structure that consists of a collection of nodes (vertices) connected by edges.

Graphs can be used to model a wide variety of real-world systems, such as social networks, transportation networks, and computer networks.



Structures

- Structures (also called structs) are a way to group several related variables into one place.
- Each variable in the structure is known as a member of the structure.
- Unlike an array, a structure can contain many different data types (int, float, char, etc.).

Create a Structure

You can create a structure by using the struct keyword and declare each of its members inside curly braces:

```
struct MyStructure
{   // Structure declaration
  int myNum;          // Member (int variable)
  char myLetter;       // Member (char variable)
}; // End the structure with a semicolon
```

Access Structure Members

To access the structure, you must create a variable of it.
Use the struct keyword inside the main() method, followed by the name of the structure and then the name of the structure variable:
To access members of a structure, use the dot syntax (.):

```
struct myStructure {                  // Create a structure called myStructure
  int myNum;
  char myLetter;
};
int main() {
  // Create a structure variable of myStructure called s1
  struct myStructure s1;

  // Assign values to members of s1
  s1.myNum = 13;
  s1.myLetter = 'B';

  // Print values
  printf("My number: %d\n", s1.myNum);
  printf("My letter: %c\n", s1.myLetter);

  return 0;
}
```

Example 2:

```
struct Car {
  char brand[50];
  char model[50];
  int year;
};

int main() {
  struct Car car1 = {"BMW", "X5", 1999};
  struct Car car2 = {"Ford", "Mustang", 1969};
  struct Car car3 = {"Toyota", "Corolla", 2011};

  printf("%s %s %d\n", car1.brand, car1.model, car1.year);
  printf("%s %s %d\n", car2.brand, car2.model, car2.year);
  printf("%s %s %d\n", car3.brand, car3.model, car3.year);

  return 0;
}
```

Assessment Question

WAP for the below given scenario:

• A university wants to automate their admission process. Students are admitted based on the marks scored in a qualifying exam.

• A student is identified by student id, age and marks in qualifying exam. Data are valid, if:

• Age is greater than 20

• Marks is between 0 and 100 (both inclusive)

• A student qualifies for admission, if

• Age and marks are valid and Marks is 65 or more

• Write a program to represent the students seeking admission in the university.

Pointers

- A pointer is a variable that stores the memory address of another variable as its value.
- A pointer variable points to a data type (like int) of the same type, and is created with the * operator.
- The address of the variable you are working with is assigned to the pointer:
- You can also get the value of the variable the pointer points to, by using the * operator (the dereference operator):

Example

```
int myAge = 43;     // Variable declaration
int* ptr = &myAge;  // Pointer declaration

// Reference: Output the memory address of myAge with the pointer (0x7ffe5367e044)
printf("%p\n", ptr);

// Dereference: Output the value of myAge with the pointer (43)
printf("%d\n", *ptr);
```

- WAP to swap two numbers using functions

```c
#include <stdio.h>

// function to swap the two numbers
void swap(int *x,int *y)
{
    int t;
     t   = *x;
    *x   = *y;
    *y   =  t;
}
```

```c
int main()
{
    int num1,num2;

    printf("Enter value of num1: ");
    scanf("%d",&num1);
    printf("Enter value of num2: ");
    scanf("%d",&num2);

    //displaying numbers before swapping
    printf("Before Swapping: num1 is: %d, num2 is: %d\n",num1,num2);

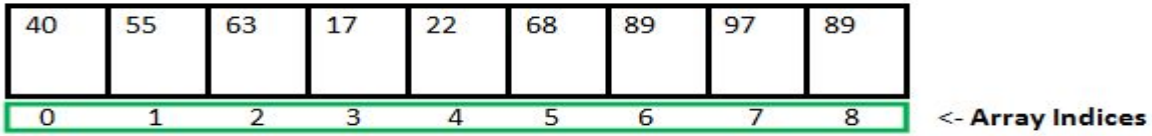
    //calling the user defined function swap()
    swap(&num1,&num2);

    //displaying numbers after swapping
    printf("After  Swapping: num1 is: %d, num2 is: %d\n",num1,num2);

    return 0;
}
```

Dynamic Memory Allocation in C

Since C is a structured language, it has some fixed rules for programming. One of them includes changing the size of an array. An array is a collection of items stored at contiguous memory locations.



Array Length = 9
First Index = 0
Last Index = 8

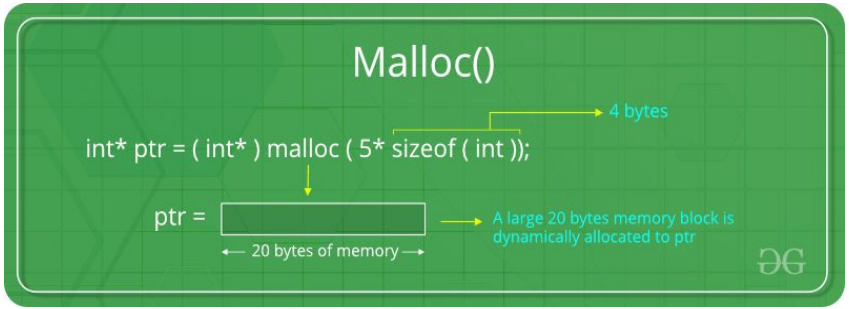
- If there is a situation where only 5 elements are needed to be entered in this array. In this case, the remaining 4 indices are just wasting memory in this array. So there is a requirement to lessen the length (size) of the array from 9 to 5.
- Take another situation. In this, there is an array of 9 elements with all 9 indices filled. But there is a need to enter 3 more elements in this array. In this case, 3 indices more are required. So the length (size) of the array needs to be changed from 9 to 12.

This procedure is referred to as **Dynamic Memory Allocation in C**.
Therefore, C **Dynamic Memory Allocation** can be defined as a procedure in which the size of a data structure (like Array) is changed during the runtime.
C provides some functions to achieve these tasks. There are 4 library functions provided by C defined under **<stdlib.h>** header file to facilitate dynamic memory allocation in C programming. They are:

1. malloc()
2. calloc()
3. free()
4. realloc()

C malloc() method

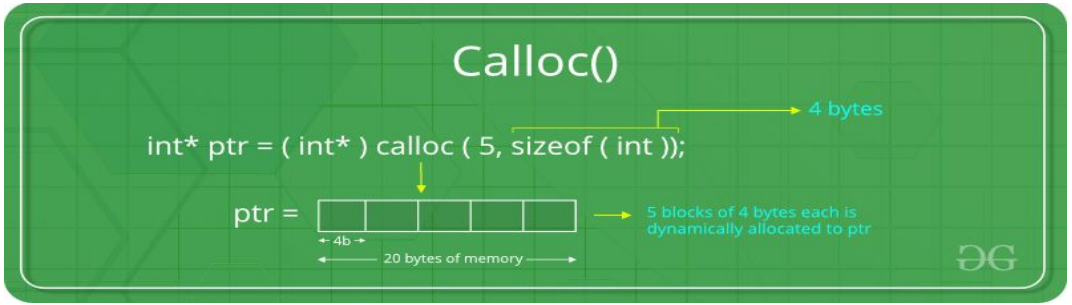
The **“malloc”** or **“memory allocation”** method in C is used to dynamically allocate a single large block of memory with the specified size. It returns a pointer of type void which can be cast into a pointer of any form. It doesn’t Initialize memory at execution time so that it has initialized each block with the default garbage value initially. If space is insufficient, allocation fails and returns a NULL pointer.



C calloc() method **“calloc”** or **“contiguous allocation”** method in C is used to dynamically allocate the specified number of blocks of memory of the specified type. it is very much similar to malloc() but has two different points and these are:

1. It initializes each block with a default value ‘0’.
2. It has two parameters or arguments as compare to malloc().

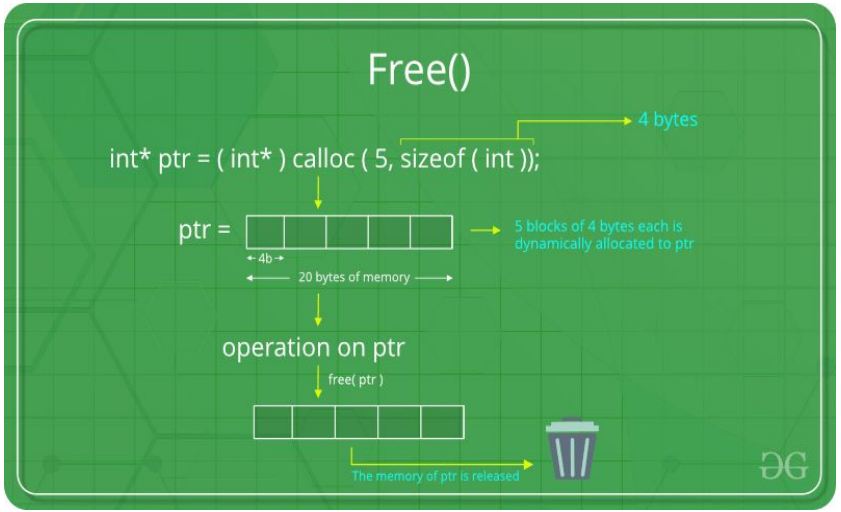
If space is insufficient, allocation fails and returns a NULL pointer.



C free() method

“free” method in C is used to dynamically **de-allocate** the memory. The memory allocated using functions malloc() and calloc() is not de-allocated on their own. Hence the free() method is used, whenever the dynamic memory allocation takes place. It helps to reduce wastage of memory by freeing it.

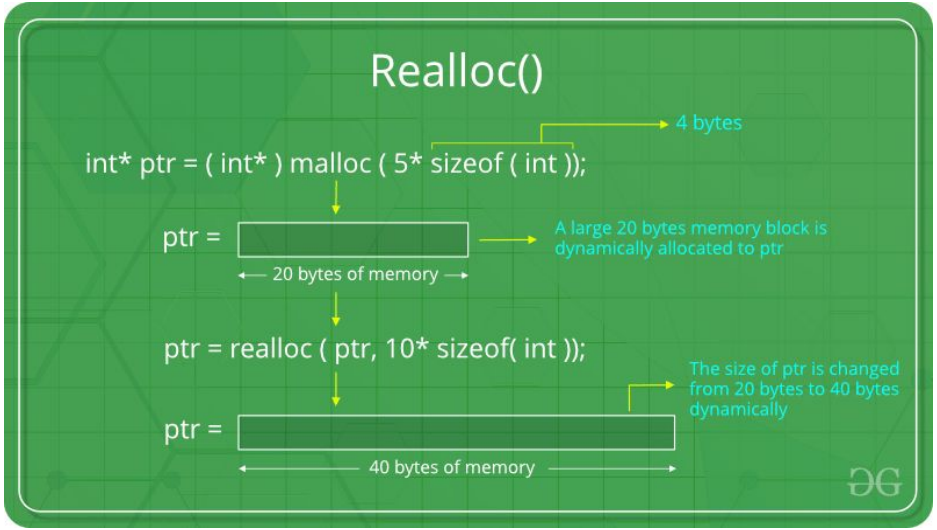
Syntax of free() in C `free(ptr);`



C realloc() method

“realloc” or **“re-allocation”** method in C is used to dynamically change the memory allocation of a previously allocated memory. In other words, if the memory previously allocated with the help of malloc or calloc is insufficient, realloc can be used to **dynamically re-allocate memory**. re-allocation of memory maintains the already present value and new blocks will be initialized with the default garbage value.

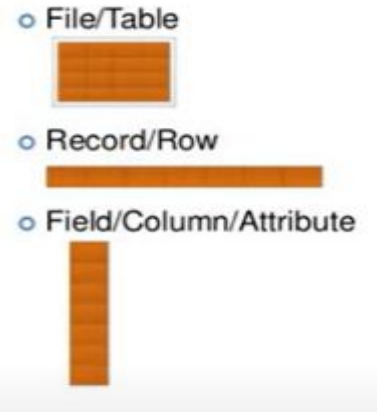
If space is insufficient, allocation fails and returns a NULL pointer.



Operations on Data Structures

- Traversing
- Insertion
- Deletion
- Searching
- Sorting
- Merging

File Organization



File:

- File is a collection of records related to each other.

File Organization:

- File organization refers to the way data is stored in a file.
- Logical relationship among various records, constituting a file.
- File organization helps in optimal design of file records so that they can be retrieved as quickly as possible.
- Improves the performance of file operations such as insert, update and delete.
- Ensures the data consistency and avoids duplicates during file operations.
- Provides optimal storage for managing the files.

Types of File Organization

s Sequential file	Indexed Files	Relative Files
These files can be accessed only sequentially.	These files can be accessed sequentially as well as randomly with the help of the record key.	These files can be accessed sequentially as well as randomly with the help of their relative record number.
The records are stored sequentially.	The records are stored based on the value of the RECORD-KEY which is the part of the data.	The records are stored by their relative address.
Records cannot be deleted and can only be stored at the end of the file.	It is possible to store the records in the middle of the file.	The records can be inserted at any given position.
It occupies less space as the records are stored in continuous order.	It occupies more space.	It occupies more space.

Sequential file	Indexed Files	Relative Files
It provides slow access, as in order to access any record all the previous records are to be accessed first.	It also provides slow access(but is fast as compared to sequential access) as it takes time to search for the index.	It provides fast access as provides the record key compared to the other two.
In Sequential file organization, the records are read and written in sequential order.	In Indexed file organization, the records are written in sequential order but can be read in sequential as well as random order.	In Relative file organization, the records can be written and read in sequential as well as random order.
There is no need to declare any KEY for storing and accessing the records.	One or more KEYS can be created for storing and accessing the records.	Only one unique KEY is declared for storing and accessing the records.

Assessment 1

Difference between Data structures and File Organization.

Stack operations

push(): **pop():**

top(): **size():** **isEmpty():** (To use a stack efficiently, we need to check the status of stack as well)

Applications

- Function calls/Recursion
- Undo operations/Editors
- Balanced parentheses/ compiler

Implementation of Stack

- Implementation of a stack using
  - Arrays
  - Linked List

Array Implementation

- Pseudo code:

```
int A[10]
top <-1 -1 //Empty stack
```

```
push(x)
{
 top<-top+1
 A[top]<-x
}
```

```
pop()
{
 top<- top - 1
}
```

```
top()
{
 return A[top]
}
```

```
IsEmpty()
{
 if(top == -1)
       return True;
else
       return False;
```

```
IsFull()
{
 if(top == SIZE)
     return True;
 else
     return False;
```

Linked List Implementation

- Will be discussed in the Linked List chapter/Unit.

```c
#include<stdio.h>
#include<conio.h>

#define SIZE 10

void push(int);
void pop();
void display();

int stack[SIZE], top = -1;

void main()
{
   int value, choice;
   clrscr();
   while(1){
      printf("\n\n***** MENU *****\n");
      printf("1. Push\n2. Pop\n3. Display\n4. Exit");
      printf("\nEnter your choice: ");
      scanf("%d",&choice);
      switch(choice){
         case 1: printf("Enter the value to be insert: ");
                 scanf("%d",&value);
                 push(value);
                 break;
         case 2: pop();
                 break;
         case 3: display();
                 break;
         case 4: exit(0);
         default: printf("\nWrong selection!!! Try again!!!");
      }
   }
}
```

```c
void push(int value){
   if(top == SIZE-1)
      printf("\nStack is Full!!! Insertion is not possible!!!");
   else{
      top++;
      stack[top] = value;
      printf("\nInsertion success!!!");
   }
}

 void pop(){
    if(top == -1)
       printf("\nStack is Empty!!! Deletion is not possible!!!");
    else{
       printf("\nDeleted : %d", stack[top]);
       top--;
    }
 }

   void display(){
      if(top == -1)
         printf("\nStack is Empty!!!");
      else{
         int i;
         printf("\nStack elements are:\n");
         for(i=top; i>=0; i--)
            printf("%d\n",stack[i]);
      }
   }
```

Example / Application - Reverse a String

```
#include <stdio.h>
#include <string.h>

#define max 100
int top=-1,stack[max];

void push(char x){

    // Push(Inserting Element in stack) operation
    if(top == max-1){
        printf("stack overflow");
    }  else {
        stack[++top]=x;
    }

}
```

```
void pop(){
    // Pop (Removing element from stack)
      printf("%c",stack[top--]);
}


int main()
{
   char str[]="Hello All";
   int len = strlen(str);
   int i;

   for(i=0;i<len;i++)
        push(str[i]);

   for(i=0;i<len;i++)
      pop();
    return 0;
     }
```

Check for balanced parentheses

```c
#include<stdio.h>
#include<string.h>
#include<stdlib.h>

#define MAX 30
int top=-1;
int stack[MAX];

void push(char);
char pop();
int match(char a,char b);
int check(char []);

int main()
{
        char exp[MAX];
        int valid;
        printf("Enter an algebraic expression : ");
        gets(exp);
        valid=check(exp);
        if(valid==1)
                printf("Valid expression\n");
        else
                printf("Invalid expression\n");

                return 0;

}
```

```
int check(char exp[] )
{
        int i;
        char temp;
        for(i=0;i<strlen(exp);i++)
        {
                if(exp[i]=='(' || exp[i]='[' || exp[i]='[')
                        push(exp[i]);
                if(exp[i]==')' || exp[i]='}' || exp[i]=']')
                        if(top==-1)     /*stack empty*/
                        {
                                printf("Right parentheses are more than left parenthese
                                return 0;
                        }
                        else
                        {
                                temp=pop();
                                if(!match(temp, exp[i]))
                                {
                                        printf("Mismatched parentheses are : ");
                                        printf("%c and %c\n",temp,exp[i]);
                                        return 0;
                                }
                        }
        }
        if(top==-1) /*stack empty*/
        {
                printf("Balanced Parentheses\n");
                return 1;
        }
        else
        {
                printf("Left parentheses more than right parentheses\n");
                return 0;
        }
}/*End of main()*/
```

```
int match(char a,char b)
{
        if(a=='[' && b==']')
                return 1;
        if(a=='{' && b=='}')
                return 1;
        if(a=='(' && b==')')
                return 1;
        return 0;
}/*End of match()*/

void push(char item)
{
        if(top==(MAX-1))
        {
                printf("Stack Overflow\n");
                return;
        }
        top=top+1;
        stack[top]=item;
}/*End of push()*/

char pop()
{
        if(top==-1)
        {
                printf("Stack Underflow\n");
                exit(1);
        }
        return(stack[top--]);
}/*End of pop()*/
```

Infix, Prefix and Postfix

Infix, Postfix and Prefix notations are three but equivalent ways of writing expressions.

Infix Expression	Prefix Expression	Postfix Expression
A + B * C + D	+ + A * B C D	A B C * + D +
(A + B) * (C + D)	* + A B + C D	A B + C D + *
A * B + C * D	+ * A B * C D	A B * C D * +
A + B + C + D	+ + + A B C D	A B + C + D +

Advantages of Postfix Expression over Infix Expression

• An infix expression is difficult for the machine to know and keep track of precedence of operators.

On the other hand, a postfix expression itself determines the precedence of operators (as the placement of operators in a postfix expression depends upon its precedence).

• Therefore, for the machine it is easier to carry out a postfix expression than an infix expression.

Infix to Postfix Conversion Algorithm

1. Print operands as they arrive.
2. If stack is empty or contains a left parenthesis on top, push the incoming operator onto the stack.
3. If incoming symbol is ‘(‘, push it onto the stack.
4. If incoming symbol is ‘)’, pop the stack and print the operator until the left parentheses is found.
5. If incoming symbol has higher precedence than the top of the stack, push it onto the stack.
6. If incoming symbol has lower precedence than the top of the stack, pop and print the top. Then test the incoming operator with the next top of the stack.

Continue...

7. If the operator has equal precedence with the top of the stack, use associativity rule.

- L to R, then pop and print the top of the stack and then push the incoming operator.
- R to L,then push the incoming operator.

8. At the end of the expression, pop and print all operator of stack.

Infix Expression: **A+ (B*C-(D/E^F)*G)*H**, where **^** is an exponential operator.

Symbol	Scanned	STACK	Postfix Expression	Description
1.		(		Start
2.	A	(	A	
3.	+	(+	A	
4.	(	(+(	A	
5.	B	(+(	AB	
6.	*	(+(*	AB	
7.	C	(+(*	ABC	
8.	-	(+(-	ABC*	'*' is at higher precedence than '-'
9.	(	(+(-(	ABC*	
10.	D	(+(-(	ABC*D	
11.	/	(+(-(/	ABC*D	
12.	E	(+(-(/	ABC*DE	
13.	^	(+(-(/^	ABC*DE	
14.	F	(+(-(/^	ABC*DEF	
15.	)	(+(-	ABC*DEF^/	Pop from top on Stack, that's why '^' Come first
16.	*	(+(-*	ABC*DEF^/	
17.	G	(+(-*	ABC*DEF^/G	
18.	)	(+	ABC*DEF^/G*-	Pop from top on Stack, that's why '^' Come first
19.	*	(+*	ABC*DEF^/G*-	
20.	H	(+*	ABC*DEF^/G*-H	
21.	)	Empty	ABC*DEF^/G*-H*+	END

Implementation of infix to postfix

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX 100 // Define a maximum size for the arrays

// Function to return precedence of operators
int prec(char c) {
    if (c == '^')
          return 3;
    else if (c == '/' || c == '*')
         return 2;
    else if (c == '+' || c == '-')
        return 1;
    else
        return -1; }

// Function to return associativity of operators
char associativity(char c) {
  if (c == '^')
     return 'R';
  return 'L'; // Default to left-associative
}
```

```
// The main function to convert infix expression to postfix expression
void infixToPostfix(const char *s) {
        char result[MAX];
        char stack[MAX];
        int resultIndex = 0;
        int stackIndex = -1;
        int len = strlen(s);
        for (int i = 0; i < len; i++) {
        char c = s[i];
        // If the scanned character is an operand, add it to the output string.
        if ((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || (c >= '0' && c <= '9')) {
          result[resultIndex++] = c;
        } else if (c == '(') {
        // If the scanned character is an ‘(‘, push it to the stack.
```

```
            stack[++stackIndex] = c;
        } else if (c == ')') {
            // If the scanned character is an ‘)’, pop and add to the output string from the stack
until an ‘(‘ is encountered.
                while (stackIndex >= 0 && stack[stackIndex] != '(') {
                result[resultIndex++] = stack[stackIndex--];
            }
            stackIndex--; // Pop '('
        }
else {
                // If an operator is scanned
                while (stackIndex >= 0 && (prec(c) < prec(stack[stackIndex]) ||
                   (prec(c) == prec(stack[stackIndex]) && associativity(c) == 'L'))) {
                result[resultIndex++] = stack[stackIndex--];
                }
            stack[++stackIndex] = c;
        }
}
```

```
    // Pop all the remaining elements from the stack
        while (stackIndex >= 0) {
        result[resultIndex++] = stack[stackIndex--];
        }

    result[resultIndex] = '\0';   // Null-terminate the result
    printf("Postfix expression: %s\n", result);
}

int main() {
        char exp[MAX]; // Fixed-size array for user input

        printf("Enter an infix expression: ");
        fgets(exp, MAX, stdin);
        exp[strcspn(exp, "\n")] = 0; // Remove trailing newline

    infixToPostfix(exp);
        return 0;
}
```

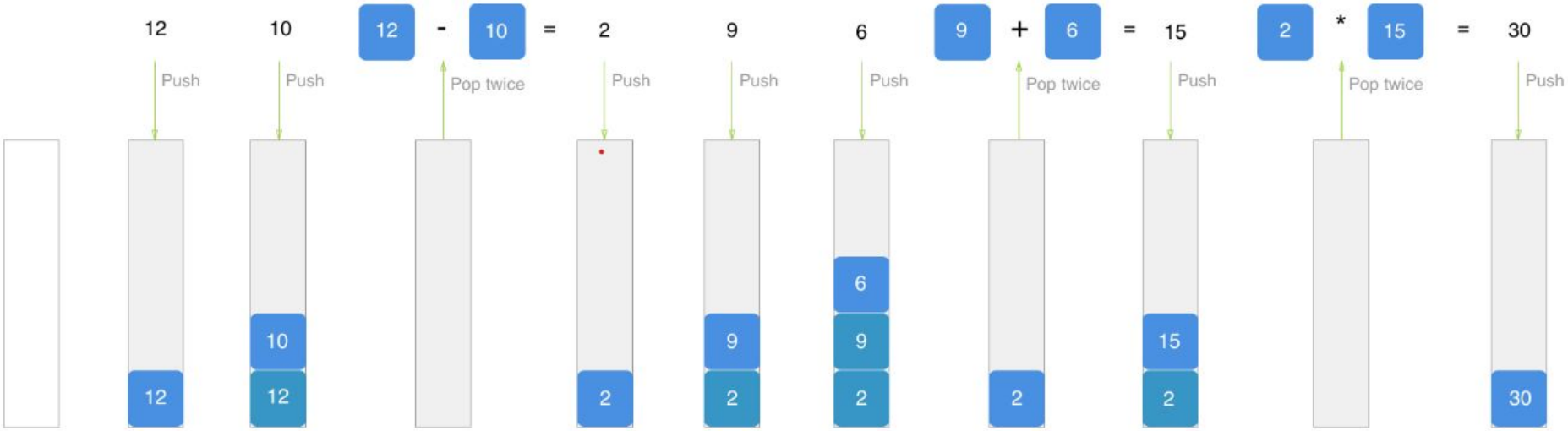
Evaluation of Postfix Expression

Algorithm for Postfix Evaluation

1. Create an empty stack called operandStack.
2. Scan the string from left to right.
    - If the token is an operand, push the value onto the operand stack.
    - If the token is an operator, *, /, +, or -, it will need two operands. Pop the operandStack twice. The first pop is the second operand and the second pop is the first operand. Perform the arithmetic operation. Push the result back on the operand stack.
3. When the input expression has been completely processed, the result is on the stack. Pop the operand stack and return the value.

Postfix Evaluation Example

Example : 12 10 - 9 6 + *



Examples

1. 3 4 + 2 * 7 /
2. 2 3 1 * + 9 -
3. 100 200 + 2 / 5 * 7 +

Recursion

- The process in which a function calls itself directly or indirectly is called recursion and the corresponding function is called a recursive function.
- Using a recursive algorithm, certain problems can be solved quite easily. Examples of such problems are Tower of Honai(TOH), Inorder/preorder/Postorder Tree Traversals, DFS of Graph, etc.

Properties of Recursion:

- Performing the same operations multiple times with different inputs.
- Base condition is needed to stop the recursion otherwise infinite loop will occur.
- The recursive function uses LIFO (LAST IN FIRST OUT) Structure just like the stack data structure.

Algorithm: Steps

The algorithmic steps for implementing recursion in a function are as follows:

Step1 - Define a base case: Identify the simplest case for which the solution is known or trivial. This is the stopping condition for the recursion, as it prevents the function from infinitely calling itself.

Step2 - Define a recursive case: Define the problem in terms of smaller subproblems. Break the problem down into smaller versions of itself, and call the function recursively to solve each subproblem.

Step3 - Ensure the recursion terminates: Make sure that the recursive function eventually reaches the base case, and does not enter an infinite loop.

Step4 - Combine the solutions: Combine the solutions of the subproblems to solve the original problem.

Let us consider a problem to determine the sum of first n natural numbers, there are several ways of doing that but the simplest approach is simply to add the numbers starting from 1 to n. So the function simply looks like this,

approach(1) – Simply adding one by one

f(n) = 1 + 2 + 3 +.....+ n

but there is another mathematical approach of representing this,

approach(2) – Recursive adding

f(n) = 1 n=1

f(n) = n + f(n-1) n>1

In **approach(2)** the function " **f( )** " itself is being called inside the function, so this phenomenon is named recursion, and the function containing recursion is called recursive function, at the end, this is a great tool in the hand of the programmers to code some problems in a lot easier and efficient way.

In the recursive program, the solution to the base case is provided and the solution to the bigger problem is expressed in terms of smaller problems.

Example: Find a Factorial of n

• factorial of n (n!) = 1 * 2 * 3 * 4 *... * n

```
int fact(int n)
{
    if(n<=1)    //base case
        return 1;
     else
        return n * fact(n-1);
}
```

In the above example, the base case for n < = 1 is defined and the larger value of a number can be solved by converting to a smaller one till the base case is reached.

Example: Fibonacci series

• 0 1 1 2 3 5 8 13 21 ....
Fib(n) = Fib(n-1) + Fib(n-2)

```
int fib(int n)
{
   if(n<=1)
        return n;

   else
       return fib(n-1) + fib(n-2);
}
```

Example: Indirect Recursion

What is the difference between direct and indirect recursion?

A function fun is called direct recursive if it calls the same function fun. A function fun is called indirect recursive if it calls another function say fun_new and fun_new calls fun directly or indirectly.

• Print odd number to n+1 and even number to n-1 (if n=10, print 2,1,4,3,6,5,8,7,10,9)

```
func odd(int n)
{
  if (n<=10)
   {
     printf("%d",n+1);
     n++;
     even(n);
   }
return;
}
```

```
func even(int n)
{
  if (n<=10)
   {
     printf("%d",n-1);
     n++;
     odd(n);
   }
return;
}
```

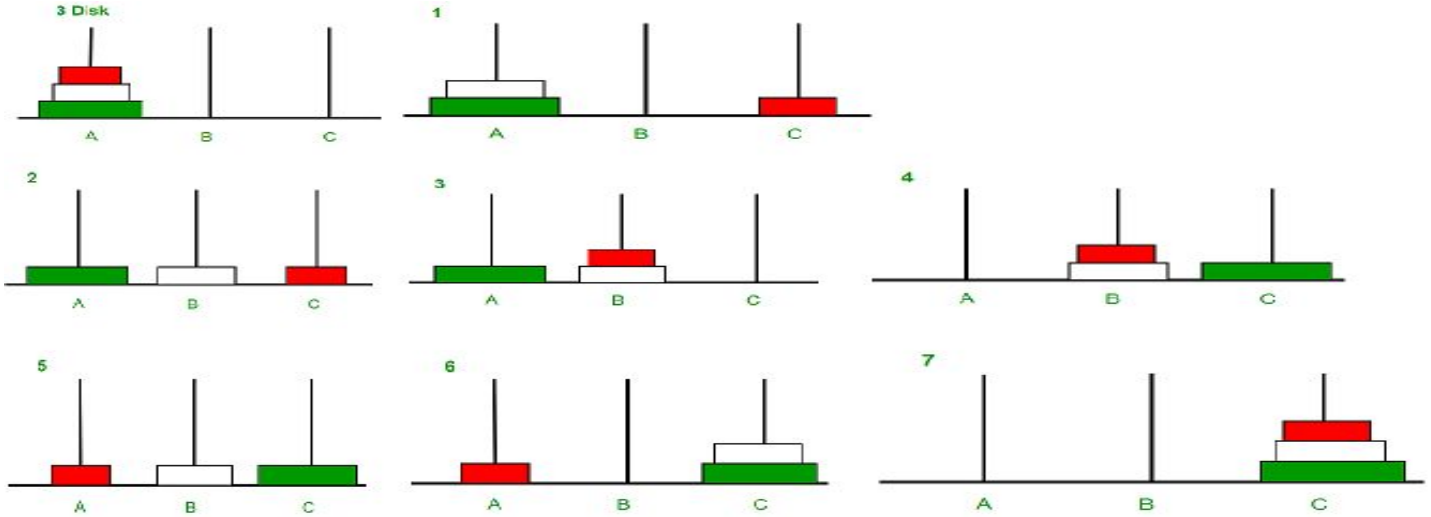
Example: Tower of Hanoi

- Tower of Hanoi is a mathematical puzzle where we have three rods (**A**, **B**, and **C**) and **N** disks.
- Initially, all the disks are stacked in decreasing value of diameter i.e., the smallest disk is placed on the top and they are on rod **A**.
- The objective of the puzzle is to move the entire stack to another rod (here considered **C**), obeying the following simple rules:
  - Only one disk can be moved at a time.
  - Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack i.e. a disk can only be moved if it is the uppermost disk on a stack.
  - No disk may be placed on top of a smaller disk.

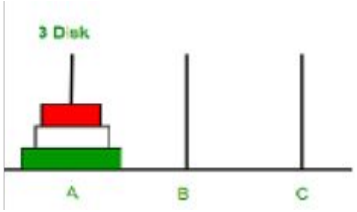
Tower of Hanoi using Recursion:

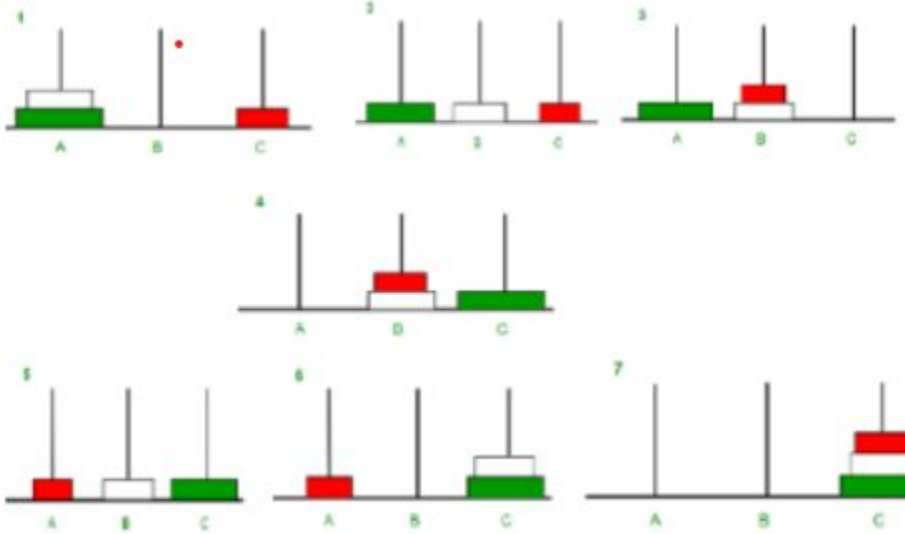
The idea is to use the helper node to reach the destination using recursion. Below is the pattern for this problem:

- *S1: Shift 'N-1' disks from 'A' to 'B', using C.*
- *S2: Shift last disk from 'A' to 'C'.*
- *S3: Shift 'N-1' disks from 'B' to 'C', using A.*



- *S1: Shift ‘N-1’ disks from ‘A’ to ‘B’, using C.*
- *S2: Shift last disk from ‘A’ to ‘C’.*
- *S3: Shift ‘N-1’ disks from ‘B’ to ‘C’, using A.*





<-S1

<-S2

<-S3

Follow the steps below to solve the problem:

- Create a function **towerOfHanoi** where pass the **N** (current number of disk), **from_rod**, **to_rod**, **aux_rod.**
- Make a function call for N – 1 th disk.
- Then print the current the disk along with **from_rod** and **to_rod**
- Again make a function call for N – 1 th disk.

Pseudocode:

```
Towerofhanoi(n,s,a,d)
{
   if(n==1)
   {
        print “S ->D”
        return;
   }
   Towerofhanoi(n-1,s,d,a)
    printf “S->D”
    Towerofhanoi(n-1,a,s,d)
}
```

Time complexity: O(2^N), There are two possibilities for every disk. Therefore, 2 * 2 * 2 * . . . * 2(N times) is 2^N
Auxiliary Space: O(N), Function call stack space

Trace the code

Code tracing is a method in which the programmer uses paper and pencil to hand trace the execution of a program or code segment in order to track the variable values as they change during execution and to determine the output of the code.

1.

```
int sum(int n)
{
        if (n ==1)
           return 1;
        else
            return n+ sum(n-1);
}
```

1. the sum of first n natural numbers, In this **approach** the function “ **f( )** ” itself is being called inside the function, so this phenomenon is named recursion,

2. Trace the code

```
#include <stdio.h>
int main() {

  int x[] = {10, 20, 30};

  int* p = &x[1];

  char* fruit[3] = {"apples",

                    "bananas",

                    "cherries"};

  printf("I have %d %s\n", *p, fruit[1]);
  return 0;
}
```

2.

```
I have 20 bananas
```

```
#include <stdio.h>
int main() {

  int x[] = {10, 20, 30};

  int* p = &x[1]; // pointer into middle

  char* fruit[3] = {"apples",

                    "bananas",

                    "cherries"};

  printf("I have %d %s\n", *p, fruit[1]);
  return 0;
}
```

